

CPSI 28003 – 01 and 9S1 – Algorithms

Take-Home Midterm Exam (Open Book and Note)

10:50 AM – 12:15 PM
Tuesday, October 14, 2025

(Please submit the midterm to the Blackboard.)

Department of Computer Science
University of Arkansas at Little Rock
2801 South University Ave.
Little Rock, Arkansas 72204-1099

Total Points: 100 Points

Total Pages: 8

Name: Isaac Fredricks T#: 2765 (Last 4 digits)

Part I: Multiple Choice Questions (10 Points)

1. ^d _____ (2 Points) If a variable is a pointer to a data type, then which of the following operator is used for dereferencing?
 - a) $\rightarrow$
 - b) $.$
 - c) $\&$
 - d) $*$
2. ^b _____ (2 Points) `int* x , y;`
 - a) x is an integer type and y is a pointer to int
 - b) x is a pointer to int and y is an integer type;
 - c) x and y both are pointer to int
 - d) illegal declaration statement
3. ^b _____ (2 Points) Which one of the following is **illegal**?
 - a) `int i; int* ip = &i;`
 - b) `int x[5]; int* pA = x;`
 - c) `int y[5]; int* pY = y[3];`
 - d) `int* ip1; ip1 = 0;`

4. b (2 Points) What is the output of the following C/C++ statements?

```
void main()
{
    int ary[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
    int *p = ary+1;
    int *q = ary+6;
    printf("%d\n", q-p);
}
```

- a) 6
- b) 5
- c) 9
- d) 4

5. c (2 Points) What is the output when the following program runs?

```
char strPtr[] = "abcdefg";
cout << strPtr + 5;
```

- a) g
- b) ef
- c) fg
- d) efg

Part II: Very Short Answers (10 Points)

1. (10 Points) Given a sorted array of n integers,

```
bool XYZ(int ary[], int low, int high, int key)
{
    if (high >= low)
    {
        int mid = low + (high - low) / 2;

        if (ary[mid] == key)
            return true;

        if (ary[mid] > key)
            return XYZ(ary, low, mid - 1, key);
        else
```

```

        return XYZ(ary, mid + 1, high, key);
    }
    return false;
}

```

The following program drives XYZ.

```

int main()
{
    int arr[] = {-50, -40, -30, -20, -10, 11, 21, 30, 41, 51};

    int n = sizeof(arr) / sizeof(arr[0]);
    for (int i = 0; i < n-2; i++)
    {
        bool result = XYZ(arr, 0, n-1, -arr[i]);
        if (result == true)
        {
            cout << "A pair is found in the array!\n";
            break;
        }
    }

    system("PAUSE");
    return 0;
}

```

What is the big-O of the main() function?

[ANSWER] **No need to show the work.**

$O(n^2)$

Part III: Short Answers (30 Points)

1. (10 Points) What is the time complexity of the following C code in Big-O? **Show the work.**

```

int fun (int n)
{
    int count = 0 ;
    for (int i =n ; i>0 ; i/=2)
        for (int j=0 ; j<i ; j++)
            count++ ;
    return count ;
}

```

[ANSWER]

$O(n \log n)$

1.

	0	1	2	...	n
i	0	1	2	...	$\log(n)$
j	0	1	2	...	$n-1$
# of exec.	0	1	4	...	$(n-1) \log n$

$$f(n) = (n-1)(\log n) \in O(n \log n)$$

2. (15 Points) Find the complexity in Big-O notation for each of the following running time estimates in terms of the input size, n . **You don't need to show your work.**

(a) (5 Points) $f(n) = \log(100n^2)$
[ANSWER]

$O(n^2)$

(b) (5 Points) $20n^3 + 10n \log n + 5$
[ANSWER]

$O(n^3)$

(c) (5 Points) $f(n) = 2^{n+1} + 3^{n+1}$
[ANSWER]

$O(3^n)$

3. (5 Points) Find the running time complexity of the algorithm in Big-O notation. **You don't need to show your work.**

```
for (S=1; S<=9; S++)
{
  for (E=0; E<=9; E++)
  {
    for (N=0; N<=9; N++)
    {
      for (D=0; D<=9; D++)
      {
        for (M=1; M<=9; M++)
        {
          Big-O(1) statements;
        }
      }
    }
  }
}
```

[ANSWER]

$O(1)$

Part IV: Long Answers (35 Points)

1. (10 Points) **(Medium)** Find the complexity in **Big-O** notation for the following running time estimates in terms of the input array size, n .

```
#include <iostream>
using namespace std;
for (int i=0; i<n; i++)
    for (int j = i+1; j<n-1; j++)
        if (Arr[i] == Arr[j]) // Arr is an array of size n
            return false;
```

[ANSWER] Show the work on how you reach the answer. $O(n^2)$

1. $i = n-1$
 $j = n-1$

i	0	1	2	...	$n-2$	$n-1$
j	0	1	...	$n-2$	$n-1$	

of
exes. $n \quad n-1 \quad \dots \quad 2 \quad 1$

$$f(n) = n + (n-1) + \dots + 2 + 1 = \frac{n(n+1)}{2} = \frac{1}{2}(n^2 + n) \in O(n^2)$$

2. (10 Points) **(Easy)** What is the **Big-O** of the following recurrence relation using the **master method**. Show the work to

$$T(n) = 5T(n/3) + O(n)$$

[ANSWER]

$$T(n) = aT(n/b) + f(n)$$

$a = 5$
 $b = 3$
 $d = 1$

$$b^d = 3 < a = 5 \quad \text{so big-O is } n^{\log_b a}$$

$$O(n^{\log_3 5})$$

3. (15 Points) **Solve the following recurrence relations for $T(n)$ in closed form. No need to compute Big-O.**

- (a) (10 Points) **(Medium)** Solve the following recurrence relation for $T(n)$ in closed form using the **forward or backward substitution**. You need to show your work to get the credits.

$$T(n) = 7T(n-1) \text{ for all } n \geq 1$$

$$T(2) = 98$$

[ANSWER]

$$O(7^n)$$

3. $T(2) = 98$ wait $k \geq 1$

$$T(3) = 7T(3-1) = 7T(2) = 7 \cdot 98 = 686$$

$$T(4) = 7T(4-1) = 7T(3) = 7 \cdot T(7T(2)) = 7^2 T(2) = 7^2 \cdot 98 = 4802$$

$$\vdots$$

$$T(n-1) = 7T(n-2) = 7T(7T(n-3)) \dots = 7^{k-1} (1 - (k-1))$$

$$T(n) = 7T(n-1) = 7T(7T(n-2)) = \dots = 7^k (1 - k)$$

$$O(7^n)$$

- (b)(5 Points) **(Easy)** Solve the following recurrence relation for $T(n)$ in closed form using the **forward or backward substitution**. **You need to show your work to get the credits.**

$$T(N) = \sqrt{T^2(N-1) + 1}, \text{ where } N \geq 1$$

$$T(0) = 1$$

[ANSWER]

If $T(n) = (n+1)^{1/2}$ then $O(n^{1/2})$

$$\begin{aligned}
 b) \quad T(0) &= 1 \\
 T(1) &= \sqrt{T^2(0) + 1} = \sqrt{1^2 + 1} = \sqrt{2} \\
 T(2) &= \sqrt{T^2(1) + 1} = \sqrt{(\sqrt{2})^2 + 1} = \sqrt{2+1} = \sqrt{3} \\
 T(3) &= \sqrt{T^2(2) + 1} = \sqrt{(\sqrt{3})^2 + 1} = \sqrt{3+1} = \sqrt{4} = 2 \\
 &\vdots \\
 T(n-1) &= \sqrt{T^2(n-2) + 1} = \sqrt{T^2(n-3) + 1} = \dots = \sqrt{T^2(0) + 1} = \sqrt{1^2 + 1} = \sqrt{2} \\
 T(n) &= \sqrt{T^2(n-1) + 1} = \dots = \sqrt{(\sqrt{n+1})^2 + 1} = \sqrt{n+1+1} = \sqrt{n+2}
 \end{aligned}$$

Part V: Programming Code (15 Points)

1. (5 Points) Assume that SLL is a singly linked list with the head node (sample SLL shown below). The declarations of the Node and singlyLinkedList are provided below. The head always points to the first node of SLL. If SLL is empty, the head will be NULL. **Assume that each node contains a unique integer in SLL.**

```

class Node {
friend class singlyLinkedList;

public:
    Node();
    Node(int);
    ~Node();
    int getValue();
private:
    int value;
    Node *next;
};

class singlyLinkedList {
public:
    singlyLinkedList();
    ~singlyLinkedList();
    void addFront(Node *newNode);
    void swapNode(Node *tNode);
    int findSmallestInt();
    void displaySinglyLinkedList();
private:
    Node *head;
};

```

(a) Fill in the blanks. (5 Points)

```
int singlyLinkedList::findSmallestInt()
{
    Node *tempNode;
    int SmallestInt;

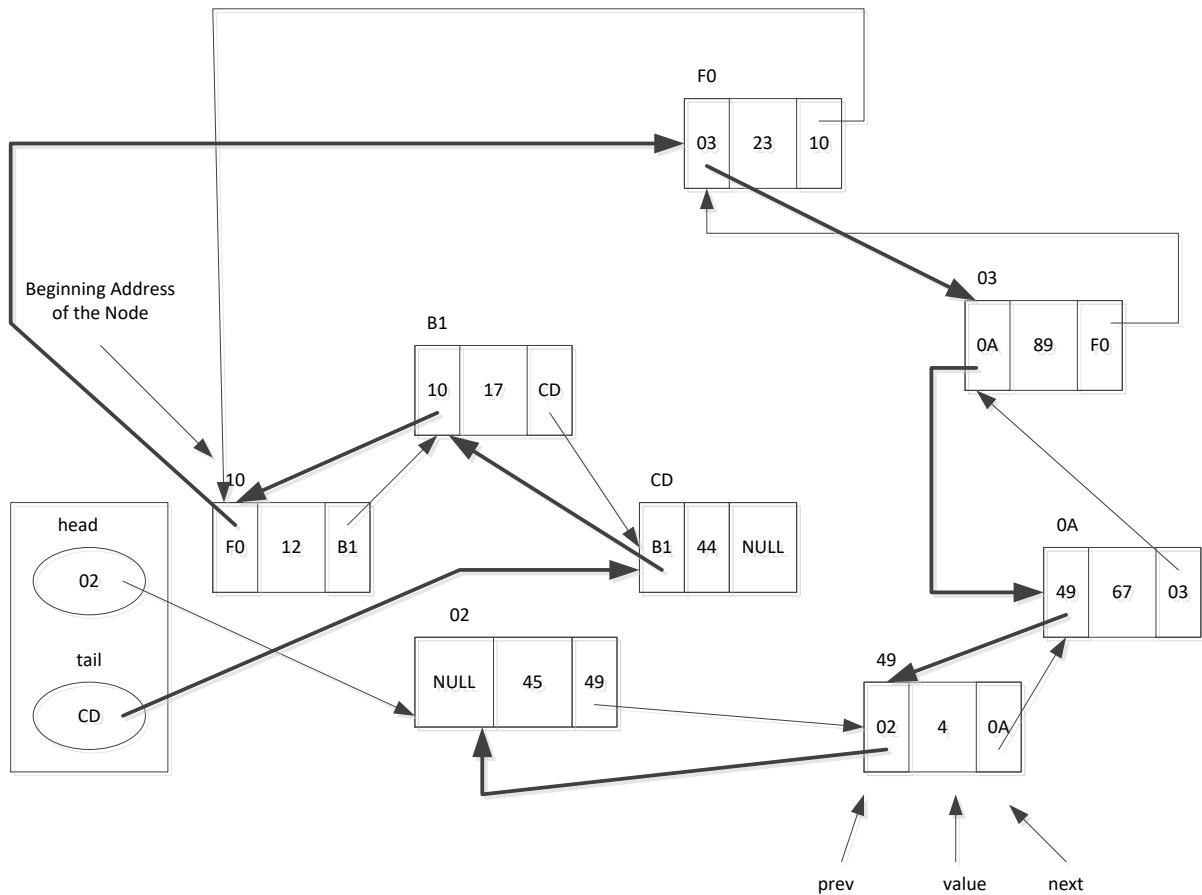
    tempNode = head;

    if (head == NULL)
    {
        cout << "Empty Singly Linked List!\n";
        exit;
    }
    else
    {
        SmallestInt = tempNode->next;
        while (tempNode != NULL)
        {
            if (tempNode->value < SmallestInt->value)
                SmallestInt = tempNode->value;
            tempNode = tempNode->next;
        }
        return (SmallestInt->value);
    }
}
```

2. (10 Points) A **doubly linked list** and the values of the links are given below. The arrow in each node indicates that the address of the node to which it is pointing is stored in that node. For example, the first node is at memory location 02 in hex, and the second node is at memory location 49 in hex. The last node is at the memory location CD in hex.

Given the Node.h, a doubly linked list is created below,

```
class Node {
    ...
    ...
private:
    Node *prev;
    int value;
    Node *next;
};
```



Fill in the blanks.

Statement	Value
cout << head→next→value;	45
cout << head→prev;	NULL
cout << head→next→next;	0A
cout << tail→prev→value;	17
cout << tail→prev→prev→prev;	F0